

UNIT - I

Practice – 1.1

Aim:

To design and implement different collision handling techniques used in hashing such as Linear Probing, Quadratic Probing, and Separate Chaining, and to study their time and space complexities.

Description:

Hashing is a technique used to store and retrieve data efficiently using a hash function. The hash function converts a key into an index of a hash table where the value is stored. Sometimes two different keys may generate the same index. This situation is called a collision.

To handle collisions, different resolution techniques are used.

1. Separate Chaining (Open Hashing) :

In Separate Chaining, each index of the hash table stores a linked list. If multiple keys hash to the same index, they are inserted into the linked list at that position. This method avoids overwriting existing elements and allows multiple elements to be stored at the same index.

Advantages:

Easy to implement

Reduces clustering problem

Time Complexity:

Average Case: $O(1)$

Worst Case: $O(n)$

Space Complexity: $O(n)$

2. Linear Probing :

Linear probing is a type of open addressing technique where collisions are resolved by checking the next available slot in the table.

If a collision occurs at index h , the algorithm checks:

$h + 1$

$h + 2$

$h + 3$

until an empty position is found.

However, linear probing may cause primary clustering, where consecutive filled slots appear in the table.

Time Complexity:

Average Case: $O(1)$

Worst Case: $O(n)$

Space Complexity: $O(n)$

3. Quadratic Probing:

Quadratic probing is another collision resolution technique that reduces clustering.

Instead of checking consecutive slots, the probing sequence follows a quadratic function

The next position is calculated using:

$\text{index} + i^2$

where i is the probe number.

This spreads the elements more evenly across the table and improves performance compared to linear probing.

Time Complexity:

Average Case: $O(1)$

Worst Case: $O(n)$

Space Complexity: $O(n)$

Procedure / Algorithm with Complexities:

Algorithm for Hashing with Collision Resolution

1. Start the program.
2. Define the size of the hash table.
3. Create a hash function to compute the index using $\text{key \% table size}$.
4. Ask the user to select the hashing technique.
5. For insertion, compute the hash index.
6. If the position is empty, insert the key.
7. If a collision occurs, apply the selected technique:
 - Linear Probing
 - Quadratic Probing
 - Separate Chaining
8. Provide options for insertion, deletion, searching and displaying the table.
9. Repeat the process until the user exits the program.
10. Stop.

Time Complexity:

Best Case: $O(1)$

Worst Case: $O(n)$

Space Complexity: $O(n)$

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
#define SIZE 10
#define EMPTY INT_MIN
#define DELETED (INT_MIN+1)
#define CHAIN_SIZE 11
int hashTable[SIZE];
struct Node{int data;struct Node *next;};
struct Node *chainTable[CHAIN_SIZE];
int hashFunction(int key){return key%SIZE;}
int chainHashFunction(int key){return key%CHAIN_SIZE;}
void initTable(){int i;for(i=0;i<SIZE;i++)hashTable[i]=EMPTY;}
void initChainTable(){int i;for(i=0;i<CHAIN_SIZE;i++)chainTable[i]=NULL;}
```

```

void insertLinear(int key){
int index=hashFunction(key);
int i,p;
for(i=0;i<SIZE;i++){
p=(index+i)%SIZE;
if(hashTable[p]==EMPTY||hashTable[p]==DELETED){
hashTable[p]=key;
printf("Inserted %d at index %d\n",key,p);return;}}
printf("Hash table is FULL\n");}
void searchLinear(int key){
int index=hashFunction(key);
int i,p;
for(i=0;i<SIZE;i++){
p=(index+i)%SIZE;
if(hashTable[p]==EMPTY)break;
if(hashTable[p]==key){
printf("Key %d found at index %d\n",key,p);return;}}
printf("Key not found\n");}
void deleteLinear(int key){
int index=hashFunction(key);
int i,p;
for(i=0;i<SIZE;i++){
p=(index+i)%SIZE;
if(hashTable[p]==EMPTY)break;
if(hashTable[p]==key){
hashTable[p]=DELETED;
printf("Key deleted\n");return;}}
printf("Key not found\n");}
void insertQuadratic(int key){
int index=hashFunction(key);
int i,p;
for(i=0;i<SIZE;i++){
p=(index+i*i)%SIZE;
if(hashTable[p]==EMPTY||hashTable[p]==DELETED){
hashTable[p]=key;
printf("Inserted %d at index %d\n",key,p);return;}}
printf("Hash table is FULL\n");}
void searchQuadratic(int key){

```

```

int index=hashFunction(key);
int i,p;
for(i=0;i<SIZE;i++){
p=(index+i*i)%SIZE;
if(hashTable[p]==EMPTY)break;
if(hashTable[p]==key){
printf("Key %d found at index %d\n",key,p);return;} }
printf("Key not found\n");}
void deleteQuadratic(int key){
int index=hashFunction(key);
int i,p;
for(i=0;i<SIZE;i++){
p=(index+i*i)%SIZE;
if(hashTable[p]==EMPTY)break;
if(hashTable[p]==key){
hashTable[p]=DELETED;
printf("Key deleted\n");
return;}}
printf("Key not found\n");}
void insertChain(int key){
int index=chainHashFunction(key);
struct Node *temp=chainTable[index];
while(temp){
if(temp->data==key){
printf("Duplicate key not allowed\n");return;}
temp=temp->next;}
struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
newNode->data=key;
newNode->next=chainTable[index];
chainTable[index]=newNode;
printf("Key inserted successfully\n");}
void searchChain(int key){
int index=chainHashFunction(key);
struct Node *temp=chainTable[index];
while(temp){
if(temp->data==key){
printf("Key found\n");return;}
temp=temp->next;}

```

```

printf("Key not found\n");}
void deleteChain(int key){
int index=chainHashFunction(key);
struct Node *temp=chainTable[index],*prev=NULL;
while(temp){
if(temp->data==key){
if(prev==NULL)chainTable[index]=temp->next;
else prev->next=temp->next;
free(temp);printf("Key deleted\n");return;}
prev=temp;temp=temp->next;}
printf("Key not found\n");}
void displayChain(){
int i;for(i=0;i<CHAIN_SIZE;i++){
printf("Index %d : ",i);
struct Node *temp=chainTable[i];
if(temp==NULL)printf("NULL");
while(temp){
printf("%d -> ",temp->data);
temp=temp->next;}printf("\n");}}
void displayLinear(){
int i;printf("\nHash Table\n");
for(i=0;i<SIZE;i++){
if(hashTable[i]==EMPTY)printf("[%d] EMPTY\n",i);
else if(hashTable[i]==DELETED)printf("[%d] DELETED\n",i);
else printf("[%d] %d\n",i,hashTable[i]);}}
int main(){
int methodChoice,opChoice,key;
while(1){
printf("\nSelect Hashing Method:\n1. Linear Probing\n2. Separate Chaining\n3. Quadratic
Probing\n4. Exit\nEnter choice: ");
scanf("%d",&methodChoice);
if(methodChoice==4)break;
if(methodChoice==1||methodChoice==3)initTable();
else initChainTable();
while(1){printf("\n1. Insert\n2. Delete\n3. Search\n4. Display\n5. Back\nEnter choice: ");
scanf("%d",&opChoice);
if(opChoice==5)break;
switch(opChoice){

```

```

case 1:printf("Enter key: ");scanf("%d",&key);
if(methodChoice==1)insertLinear(key);
else if(methodChoice==2)insertChain(key);
else insertQuadratic(key);break;
case 2:printf("Enter key: ");scanf("%d",&key);
if(methodChoice==1)deleteLinear(key);
else if(methodChoice==2)deleteChain(key);
else deleteQuadratic(key);break;
case 3:printf("Enter key: ");scanf("%d",&key);
if(methodChoice==1)searchLinear(key);
else if(methodChoice==2)searchChain(key);
else searchQuadratic(key);
break;case 4:if(methodChoice==1||methodChoice==3)displayLinear();
else displayChain();break;
default:printf("Invalid choice\n");}} }return 0;}

```

Input & Output:

```

C:\Users\P.ROOPA LAVANYA\ x + v
Select Hashing Method:
1. Linear Probing
2. Separate Chaining
3. Quadratic Probing
4. Exit
Enter choice: 1

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 1
Enter key: 2
Inserted 2 at index 2

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 1
Enter key: 4
Inserted 4 at index 4

1. Insert
2. Delete
3. Search
4. Display
5. Back

```

```

C:\Users\P.ROOPA LAVANYA\ x + v
Enter choice: 1
Enter key: 6
Inserted 6 at index 6

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 4

Hash Table
[0] EMPTY
[1] EMPTY
[2] 2
[3] EMPTY
[4] 4
[5] EMPTY
[6] 6
[7] EMPTY
[8] EMPTY
[9] EMPTY

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 5

```

```

C:\Users\P.ROOPA LAVANYAVI >
Enter choice: 1
Enter key: 37
Key inserted successfully

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 4
Index 0 : 55 ->
Index 1 : NULL
Index 2 : NULL
Index 3 : NULL
Index 4 : 37 ->
Index 5 : NULL
Index 6 : NULL
Index 7 : NULL
Index 8 : NULL
Index 9 : NULL
Index 10 : 43 ->

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 5
Select Hashing Method:

1. Linear Probing
2. Separate Chaining
3. Quadratic Probing
4. Exit
Enter choice: 3

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 1
Enter key: 33
Inserted 33 at index 3

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 1
Enter key: 56
Inserted 56 at index 6

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 1
Enter key: 67
Inserted 67 at index 7

1. Insert
2. Delete
3. Search
4. Display
5. Back
Enter choice: 4

Hash Table
[0] EMPTY
[1] EMPTY
[2] EMPTY
[3] 33
[4] EMPTY
[5] EMPTY
[6] 56
[7] 67
[8] EMPTY

```

Result:

Thus, the program to implement collision resolution techniques in hashing using Linear Probing, Quadratic Probing and Separate Chaining was successfully implemented and executed.

UNIT - II

Practice – 2.1

Aim:

To write and execute a program to implement AVL Tree operations such as insertion, deletion, searching and traversal (Inorder, Preorder and Postorder).

Description:

An AVL Tree is a self-balancing Binary Search Tree (BST) in which the difference between the heights of

the left subtree and the right subtree for every node is at most 1.

This difference is called the Balance Factor.

Balance Factor Formula

$$\text{Balance Factor} = \text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree})$$

For a balanced AVL Tree, the balance factor must be: [-1, 0, +1]

If the balance factor becomes greater than 1 or less than -1, the tree becomes unbalanced and rotations are performed to balance the tree.

AVL Tree Rotations:

1. Left Left (LL) Rotation

Occurs when a node is inserted in the left subtree of the left child.

2. Right Right (RR) Rotation

Occurs when a node is inserted in the right subtree of the right child.

3. Left Right (LR) Rotation

Occurs when a node is inserted in the right subtree of the left child.

4. Right Left (RL) Rotation

Occurs when a node is inserted in the left subtree of the right child.

These rotations help maintain the balanced property of AVL Trees.

Procedure / Algorithm with Complexities:

Algorithm for Insertion:

1. Start from the root node.

2. Insert the node using Binary Search Tree insertion rules.

3. Update the height of the node.

4. Calculate the balance factor.

5. If the balance factor is greater than 1 or less than -1, perform rotations:

- LL Rotation
- RR Rotation
- LR Rotation
- RL Rotation

6. Return the updated root node.

Algorithm for Deletion:

1. Locate the node to be deleted.
2. Perform standard BST deletion.
3. Update the height of nodes.
4. Calculate the balance factor.
5. Perform required rotations to maintain balance.

Algorithm for Searching:

1. Start from the root node.
2. Compare the key with the current node.
 - If equal → key found.
 - If smaller → move to the left subtree.
 - If larger → move to the right subtree.

Time Complexity:

Operation Time Complexity

Insertion $O(\log n)$

Deletion $O(\log n)$

Search $O(\log n)$

Traversal $O(n)$

Space Complexity: $O(n)$

Where n is the number of nodes in the AVL Tree.

Source Code:

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int key;
struct node *left,*right;
int height;};
int max(int a,int b){return(a>b)?a:b;}
int height(struct node *n){
return(n==NULL)?0:n->height;}
struct node* newNode(int key){
struct node *node=(struct node*)malloc(sizeof(struct node));
node->key=key;
node->left=node->right=NULL;
node->height=1;return node;}
struct node* minValueNode(struct node* node){
struct node* current=node;
```

```

while(current->left!=NULL)
current=current->left;return current;}
struct node* rightRotate(struct node* y){
struct node* x=y->left;
struct node* T2=x->right;
x->right=y;
y->left=T2;
y->height=max(height(y->left),height(y->right))+1;
x->height=max(height(x->left),height(x->right))+1;return x;}
struct node* leftRotate(struct node* x){
struct node* y=x->right;
struct node* T2=y->left;
y->left=x;
x->right=T2;
x->height=max(height(x->left),height(x->right))+1;
y->height=max(height(y->left),height(y->right))+1;return y;}
int getBalance(struct node* node){
return(node==NULL)?0:height(node->left)-height(node->right);}
struct node* insertNode(struct node* node,int key){
if(node==NULL){
printf("Node %d inserted successfully.\n",key);
return newnode(key);}
if(key<node->key)
node->left=insertNode(node->left,key);
else if(key>node->key)
node->right=insertNode(node->right,key);
else{
printf("Duplicate key %d not allowed.\n",key);return node;}
node->height=1+max(height(node->left),height(node->right));
int balance=getBalance(node);
if(balance>1&&key<node->left->key)
return rightRotate(node);
if(balance<-1&&key>node->right->key)
return leftRotate(node);
if(balance>1&&key>node->left->key){
node->left=leftRotate(node->left);
return rightRotate(node);}
if(balance<-1&&key<node->right->key){

```

```

node->right=rightRotate(node->right);
return leftRotate(node);}return node;}
struct node* deleteNode(struct node* root,int key){
if(root==NULL){printf("Key %d not found. Deletion failed.\n",key);return root;}
if(key<root->key)
root->left=deleteNode(root->left,key);
else if(key>root->key)
root->right=deleteNode(root->right,key);
else {printf("Node %d deleted successfully.\n",key);
if(root->left==NULL||root->right==NULL){
struct node* temp=root->left?root->left:root->right;
if(temp==NULL){
temp=root;
root=NULL;
}else
*root=*temp;free(temp);}
else{
struct node* temp=minValueNode(root->right);
root->key=temp->key;
root->right=deleteNode(root->right,temp->key);} }
if(root==NULL)return root;
root->height=1+max(height(root->left),height(root->right));
int balance=getBalance(root);
if(balance>1&&getBalance(root->left)>=0)
return rightRotate(root);
if(balance>1&&getBalance(root->left)<0){
root->left=leftRotate(root->left);return rightRotate(root);}
if(balance<-1&&getBalance(root->right)<=0)
return leftRotate(root);
if(balance<-1&&getBalance(root->right)>0){
root->right=rightRotate(root->right);
return leftRotate(root);}return root;}
struct node* search(struct node* root,int key){
if(root==NULL||root->key==key)return root;
if(key<root->key)
return search(root->left,key);
return search(root->right,key);}
void inorder(struct node* root){

```

```

if(root){
inorder(root->left);
printf("%d ",root->key);
inorder(root->right);} }
void preorder(struct node* root){
if(root){
printf("%d ",root->key);
preorder(root->left);
preorder(root->right);} }
void postorder(struct node* root){
if(root){
postorder(root->left);
postorder(root->right);
printf("%d ",root->key);} }
int main(){
struct node* root=NULL;
int choice,key;
while(1){
printf("\n\n--- AVL TREE MENU ---");
printf("\n1. Insert");
printf("\n2. Delete");
printf("\n3. Search");
printf("\n4. Inorder Traversal");
printf("\n5. Preorder Traversal");
printf("\n6. Postorder Traversal");
printf("\n7. Exit");
printf("\nEnter choice: ");
scanf("%d",&choice);
switch(choice){
case 1:printf("Enter key to insert: ");scanf("%d",&key);root=insertNode(root,key);break;
case 2:printf("Enter key to delete: ");scanf("%d",&key);root=deleteNode(root,key);break;
case 3:printf("Enter key to search: ");scanf("%d",&key);
if(search(root,key)) printf("Key %d FOUND in the tree.\n",key);
elseprintf("Key %d NOT FOUND in the tree.\n",key);break;
case 4:printf("Inorder Traversal: ");inorder(root);break;
case 5:printf("Preorder Traversal: ");preorder(root);break;
case 6:printf("Postorder Traversal: ");postorder(root);break;
case 7:printf("Exiting program...\n");exit(0);

```

```
default:printf("Invalid choice! Try again.\n");
```

```
}}}
```

Input & Output:

```

C:\Users\P.ROOPA LAVANYA\ x + v
--- AVL TREE MENU ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter choice: 1
Enter key to insert: 44
Node 44 inserted successfully.

--- AVL TREE MENU ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter choice: 1
Enter key to insert: 56
Node 56 inserted successfully.

--- AVL TREE MENU ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter choice: 1
Enter key to insert: 78
Node 78 inserted successfully.

```

```

C:\Users\P.ROOPA LAVANYA\ x + v
--- AVL TREE MENU ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter choice: 4
Inorder Traversal: 44 56 78

--- AVL TREE MENU ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter choice: 5
Preorder Traversal: 56 44 78

--- AVL TREE MENU ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter choice: 6
Postorder Traversal: 44 78 56

--- AVL TREE MENU ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal

```

Result:

Thus, the program to implement AVL Tree operations such as insertion, deletion, searching and traversal was successfully implemented and executed.

Practice – 2.2

Aim:

To write and execute a program to implement Max Heap and Min Heap operations such as insertion, deletion and display.

Description:

A Heap is a complete binary tree that satisfies the heap property.

There are two types of heaps:

1. Max Heap

In a Max Heap, the value of the parent node is greater than or equal to its child nodes.

Example:

50

/ \

30 40

/ \

10 20

The largest element is always at the root.

2. Min Heap

In a Min Heap, the value of the parent node is smaller than or equal to its child nodes.

Example:

10

/ \

20 30

/ \

40 50

The smallest element is always at the root.

Important Properties:

1. Heap is a Complete Binary Tree
2. Implemented using Array Representation
3. Parent index: $(i-1)/2$
4. Left child: $2*i + 1$
5. Right child: $2*i + 2$

Procedure / Algorithm with Complexities:

Algorithm for Insertion (Max Heap):

1. Insert the element at the end of the heap.
2. Compare it with its parent node.
3. If the parent is smaller, swap the elements.
4. Repeat until the heap property is satisfied.

Algorithm for Deletion (Max Heap):

1. Delete the root element.
2. Replace it with the last element.
3. Compare it with its children.
4. Swap with the largest child.

5. Repeat until the heap property is restored.

Algorithm for Insertion (Min Heap):

1. Insert element at the end.
2. Compare with the parent.
3. Swap if the parent is larger.

Algorithm for Deletion (Min Heap):

1. Remove the root element.
2. Replace it with the last element.
3. Swap with the smallest child.

Time Complexity:

Operation Complexity

Insertion $O(\log n)$

Deletion $O(\log n)$

Display $O(n)$

Space Complexity: $O(n)$

Source Code:

```
#include<stdio.h>
#define MAX 50
int h[MAX],n=0;
void swap(int*a,int*b){int t=*a;*a=*b;*b=t;}
void heapifyUp(int i){
while(i>0&&h[(i-1)/2]>h[i]){
swap(&h[i],&h[(i-1)/2]);
i=(i-1)/2;}}
void heapifyDown(int i){
int l=2*i+1,r=2*i+2,largest=i;
if(l<n&&h[l]>h[largest])largest=l;
if(r<n&&h[r]>h[largest])largest=r;
if(largest!=i){
swap(&h[i],&h[largest]);
heapifyDown(largest);}}
void insert(int v){
h[n]=v;
heapifyUp(n);
n++;}
void del(){
if(n==0){printf("Empty\n");return;}
printf("Deleted:%d\n",h[0]);
h[0]=h[--n];
heapifyDown(0);}
void display(){
for(int i=0;i<n;i++)printf("%d ",h[i]);
printf("\n");}
```

```

int main(){
int c,v;
while(1){
printf("\n1.Ins 2.Del 3.Dis 4.Exit:");
scanf("%d",&c);
switch(c){
case 1:
if(n==MAX)printf("Full\n");
else {scanf("%d",&v);insert(v);}
break;
case 2:del();break;
case 3:display();break;
case 4:return 0;
default:printf("Invalid\n");}}}

```

Input & Output:

```

C:\Users\P.ROOPA LAVANYA\
2. Delete max element
3. Display heap
4. Exit
Enter your choice: 1
Enter value to insert: 20
===== MAX HEAP MENU =====
1. Insert element
2. Delete max element
3. Display heap
4. Exit
Enter your choice: 1
Enter value to insert: 20
===== MAX HEAP MENU =====
1. Insert element
2. Delete max element
3. Display heap
4. Exit
Enter your choice: 1
Enter value to insert: 30
===== MAX HEAP MENU =====
1. Insert element
2. Delete max element
3. Display heap
4. Exit
Enter your choice: 1
Enter value to insert: 30
===== MAX HEAP MENU =====
1. Insert element
2. Delete max element
3. Display heap
4. Exit
Enter your choice: 3
----- Max Heap
30 10 20
===== MAX HEAP MENU =====
1. Insert element
2. Delete max element
3. Display heap
4. Exit

```

Result:

Thus, the program to implement Max Heap operations such as insertion, deletion, and display was successfully implemented and executed.

UNIT - III

Practice – 3.1

Aim:

To write and execute a program to sort a given list of elements using the Merge Sort technique based on the Divide and Conquer approach.

Description:

Merge Sort is an efficient, stable, comparison-based sorting algorithm that follows the Divide and Conquer strategy. In this technique, the array is repeatedly divided into two halves until each sub-array contains only one element. Then the smaller arrays are merged together in sorted order to form the final sorted array.

How it works:

Divide: The array is recursively divided into two halves until each sub-array contains a single element.

Conquer: The sub-arrays are sorted individually.

Combine: The sorted sub-arrays are merged together to produce a final sorted array.

The merging process compares elements from the two halves and places them in the correct order in a temporary array.

Procedure / Algorithm with Complexities:**Algorithm for Merge Sort:**

1. Start the program.
2. Read the number of elements n.
3. Read the array elements.
4. Divide the array into two halves using the middle index.
5. Recursively apply merge sort to the left half.
6. Recursively apply merge sort to the right half.
7. Merge the two sorted halves into a single sorted array.
8. Display the sorted array.
9. Stop.

Time Complexity:

Case Complexity

Best Case $O(n \log n)$

Average Case $O(n \log n)$

Worst Case $O(n \log n)$

Space Complexity: $O(n)$

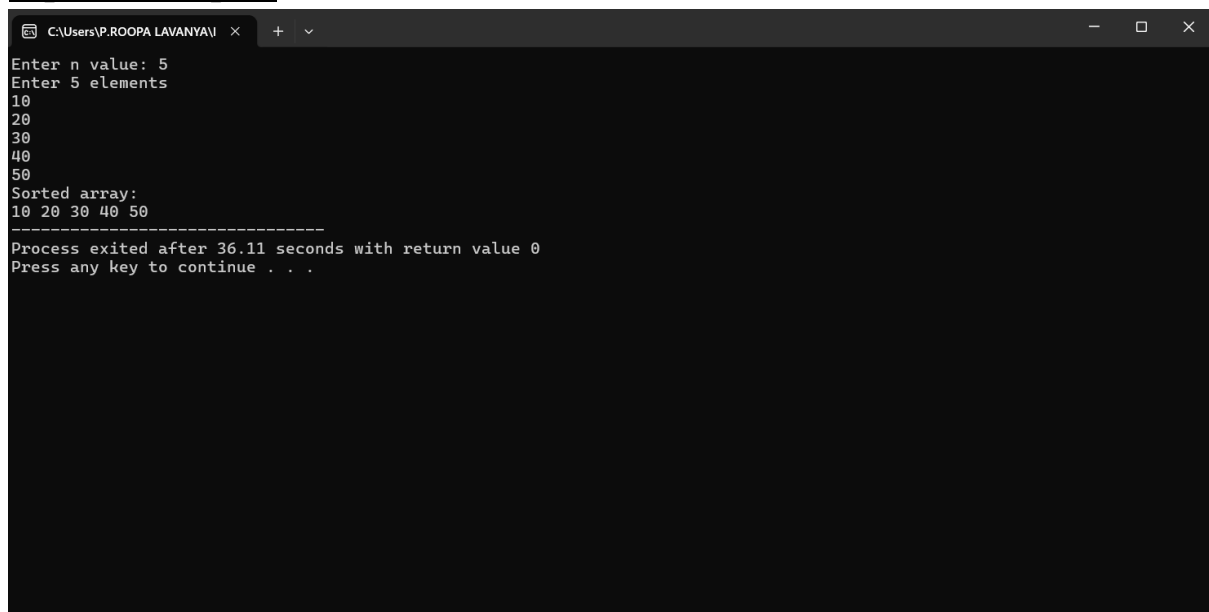
Additional memory is required for the temporary array during the merging process.

Source Code:

```
#include<stdio.h>
void merge(int a[],int l,int m,int r){
int t[50],i=l,j=m+1,k=l;
while(i<=m&&j<=r){
if(a[i]<=a[j])t[k++]=a[i++];
else t[k++]=a[j++];}
```

```
while(i<=m)t[k++]=a[i++];
while(j<=r)t[k++]=a[j++];
for(i=l;i<=r;i++)a[i]=t[i];}
void mergeSort(int a[],int l,int r){
if(l<r){
int m=(l+r)/2;
mergeSort(a,l,m);
mergeSort(a,m+1,r);
merge(a,l,m,r);}}
int main(){
int a[25],n,i;
scanf("%d",&n);
for(i=0;i<n;i++)scanf("%d",&a[i]);
mergeSort(a,0,n-1);
for(i=0;i<n;i++)printf("%d ",a[i]);
return 0;}
```

Input & Output:



```
C:\Users\P.ROOPA LAVANYA\I x + v
Enter n value: 5
Enter 5 elements
10
20
30
40
50
Sorted array:
10 20 30 40 50
-----
Process exited after 36.11 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus, the program to sort the given list of elements using the Merge Sort technique based on the Divide and Conquer approach was successfully implemented and executed, and the elements were arranged in sorted order.

Practice – 3.2

Aim:

To write and execute a C program to find the Single Source Shortest Path using the Greedy approach (Dijkstra's Algorithm).

Description:

The Single Source Shortest Path (SSSP) problem finds the shortest distance from one source vertex to all other vertices in a weighted graph. One of the most efficient algorithms to solve this problem is Dijkstra's Algorithm, which uses the Greedy approach.

Greedy Approach

The greedy method selects the best possible option at each step without considering the global solution.

In Dijkstra's Algorithm:

1. Start from the source node
2. Select the vertex with minimum distance
3. Update distances of adjacent vertices
4. Repeat until all vertices are visited

Important Properties

- Works only for graphs with non-negative edge weights
- Uses cost matrix representation
- Maintains two arrays

dist[] → shortest distance

flag[] → visited nodes

Procedure / Algorithm with Complexities:

Algorithm: Dijkstra's Algorithm

1. Read the number of nodes.
2. Read the cost matrix of the graph.
3. Initialize distance array
 $\text{dist}[i] = \text{cost}[\text{source}][i]$
4. Mark the source node as visited.
5. Repeat until all nodes are visited
6. Find the unvisited node with minimum distance
7. Mark it as visited
8. Update distances of adjacent nodes
 $\text{if}(\text{dist}[u] + \text{cost}[u][v] < \text{dist}[v])$
 $\text{dist}[v] = \text{dist}[u] + \text{cost}[u][v]$
9. Display shortest distances from the source node.

Time Complexity:

Operation Complexity

Dijkstra Algorithm $O(n^2)$

Space Complexity: $O(n)$

Where n = number of vertices.

Source Code:

```
#include<stdio.h>
#define INF 999
void dijkstra(int n,int v,int c[10][10],int d[]){
int f[10]={0},i,u,w,min,cnt=1;
for(i=0;i<n;i++)d[i]=c[v][i];d[v]=0;f[v]=1;
while(cnt<n){
min=INF;u=-1;
for(w=0;w<n;w++)
if(!f[w]&& d[w]<min){min=d[w];u=w;}
if(u==-1)break;
f[u]=1;cnt++;
for(w=0;w<n;w++)
if(!f[w]&& d[u]+c[u][w]<d[w])
d[w]=d[u]+c[u][w];} }
int main() {
int n,v,i,j,c[10][10],d[10];
scanf("%d",&n);
for(i=0;i<n;i++)
for(j=0;j<n;j++){scanf("%d",&c[i][j]);
if(c[i][j]==0)c[i][j]=INF;} scanf("%d",&v);
if(v<0||v>=n)return 0;
dijkstra(n,v,c,d);
for(i=0;i<n;i++)
if(i!=v)printf("%d->%d=%d\n",v,i,d[i]);return 0;}
```

Input & Output:

```
C:\Users\P.ROOPA LAKSHYAN\ >
Enter the number of nodes:
4
Enter the cost matrix (use 0 for no direct path):
1 0 2
2 0 1 3
0 1 3 2
1 0 3 4
Enter the source node (0 to 3):
0
Shortest path(s) from node 0:
0 -> 1, cost = 6
0 -> 2, cost = 5
0 -> 3, cost = 2
-----
Process exited after 110 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus, the program to implement Single Source Shortest Path using the Greedy approach (Dijkstra's Algorithm) was successfully implemented and executed.

UNIT - IV

Practice – 4.1

Aim:

To write a program to solve the 0/1 Knapsack problem using the Dynamic approach and find the maximum profit that can be obtained without exceeding the knapsack capacity.

Description :

The 0/1 Knapsack problem is a classical optimization problem that determines the maximum profit that can be obtained by selecting items to include in a knapsack without exceeding its capacity.

In this problem, we are given n items, each with a weight and a value (profit). It is called 0/1 Knapsack because each item can either be included (1) in the knapsack or excluded (0). Items cannot be divided.

Working Principle :

1. Initialize a DP table with 0 for all items and capacities.
2. For each item, compare including the item and excluding the item.
3. If the item weight is less than or equal to the capacity, choose the maximum profit.
4. If the item weight exceeds the capacity, exclude the item.
5. Repeat the process for all items.
6. The final cell of the table gives the maximum profit.

Applications :

- Resource allocation problems
- Cargo loading and transportation planning

Procedure / Algorithm with Complexities :

1. Start the program.
2. Read the number of items n .
3. Input the weights and values of all items.
4. Read the knapsack capacity W
5. Create a DP table $K[n+1][W+1]$ to store intermediate results.
6. Initialize the first row and first column of the table with 0.
7. For each item i (1 to n) and each capacity w (1 to W):
If the item weight is less than or equal to w , calculate the maximum of including the item or excluding the item.
☐ Otherwise, copy the value from the previous row.
8. Repeat the process until all items are processed.
9. The value stored in $K[n][W]$ represents the maximum profit.
10. Display the result and stop the program.

Time Complexity:

- $O(n \times W)$ when using a Dynamic Programming table.

Space Complexity:

- $O(n \times W)$ to store the DP table

Source Code :

```
#include<stdio.h>
int max(int a,int b){
return(a>b)?a:b;}
void knapsack(int W,int wt[],int val[],int n){
int i,w;int K[n+1][W+1];
for(i=0;i<=n;i++){for(w=0;w<=W;w++){if(i==0||w==0){K[i][w]=0;}
else if(wt[i-1]<=w){K[i][w]=max(val[i-1]+K[i-1][w-wt[i-1]],K[i-1][w]);}
else {K[i][w]=K[i-1][w];}}}
printf("Maximum profit: %d\n",K[n][W]);
printf("Items included in the knapsack:\n");w=W;
for(i=n;i>0&&K[i][w]!=0;i--){if(K[i][w]!=K[i-1][w]){
printf("Item %d (Weight: %d, Value: %d)\n",i,wt[i-1],val[i-1]);w-=wt[i-1];}}}
int main(){
int i,n,W;int val[10],wt[10];
printf("Enter the number of items: ");scanf("%d",&n);
printf("Enter the weights and values of the items:\n");
for(i=0;i<n;i++) scanf("%d %d",&wt[i],&val[i]);
printf("Enter the capacity of the knapsack: ");scanf("%d",&W);
knapsack(W,wt,val,n);return 0;}
```

Input & Output :

```
C:\Users\P.ROOPA LAVANYAU >
Enter the number of items: 5
Enter the weights and values of the items:
2 10
1 16
2 24
3 18
2 9
Enter the capacity of the knapsack: 6
Maximum profit: 58
Items included in the knapsack:
Item 4 (Weight: 3, Value: 18)
Item 3 (Weight: 2, Value: 24)
Item 2 (Weight: 1, Value: 16)
-----
Process exited after 16.77 seconds with return value 0
Press any key to continue . . .
```

Result :

Thus, the 0/1 Knapsack problem was successfully implemented using the Dynamic Programming approach, and the maximum profit for the given inputs was obtained.

Practice – 4.2

Aim :

To write a program to find the shortest path between all pairs of vertices in a graph using the Dynamic Programming approach (Floyd–Warshall Algorithm).

Description:

The All Pairs Shortest Path problem determines the shortest distances between every pair of vertices in a weighted graph. One of the most commonly used algorithms to solve this problem is the Floyd–

Warshall Algorithm, which is based on Dynamic Programming. It checks whether a shorter path exists between vertices i and j through another vertex k . If a shorter path is found, the distance matrix is updated.

This process is repeated for all vertices until the shortest distances between every pair of vertices are obtained.

Working Principle :

1. Initialize the distance matrix with the given adjacency matrix.
2. Consider each vertex as an intermediate vertex one by one.
3. Update the shortest distance between every pair of vertices if a shorter path exists through the intermediate vertex.
4. Repeat the process until all vertices are processed.
5. The final matrix represents the shortest distances between all pairs of vertices.

Applications :

- Network routing algorithms
- GPS navigation systems

Procedure / Algorithm with Complexities :

1. Start the program.
2. Read the number of vertices n .
3. Input the adjacency matrix representing the graph.
4. Initialize the distance matrix with the given graph values
5. For each vertex $k = 1$ to n , consider it as an intermediate vertex.
6. For each pair of vertices i and j , update the distance using:

$$\text{dist}[i][j] = \min (\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$
7. Repeat the process for all vertices.
8. Display the final shortest path matrix.
9. Stop the program.

Time Complexity

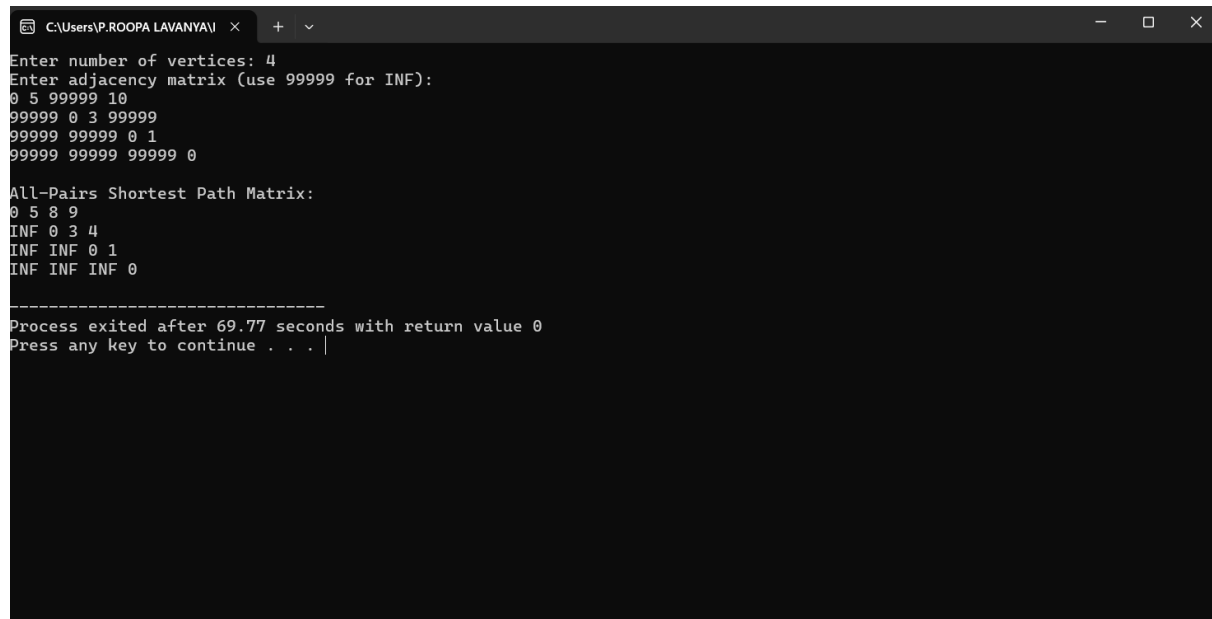
- $O(n^3)$

Space Complexity

- $O(n^2)$

Source Code :

```
#include<stdio.h>
#define INF 99999
int main(){
int n,i,j,k;printf("Enter number of vertices: ");scanf("%d",&n);
int dist[n][n];printf("Enter adjacency matrix (use %d for INF):\n",INF);
for(i=0;i<n;i++){
for(j=0;j<n;j++) scanf("%d",&dist[i][j]);
for(k=0;k<n;k++){
for(i=0;i<n;i++){
for(j=0;j<n;j++){if(dist[i][k]!=INF&&dist[k][j]!=INF&&dist[i][k]+dist[k][j]<dist[i][j])
dist[i][j]=dist[i][k]+dist[k][j];}} }printf("\nAll-Pairs Shortest Path Matrix:\n");
for(i=0;i<n;i++){
{for(j=0;j<n;j++){
if(dist[i][j]==INF)printf("INF ");
else printf("%d ",dist[i][j]);}printf("\n");}return 0;}
```

Input & Output:


```
C:\Users\P.ROOPA LAVANYA\I x + v
Enter number of vertices: 4
Enter adjacency matrix (use 99999 for INF):
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0

All-Pairs Shortest Path Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

-----
Process exited after 69.77 seconds with return value 0
Press any key to continue . . . |
```

Result:

Thus, the All Pairs Shortest Path problem was successfully implemented using the Dynamic Programming approach (Floyd-Warshall Algorithm) and the shortest distances between all pairs of vertices were obtained.

UNIT - V

Practice – 5.1

Aim :

To write a program to solve the N-Queens problem using the Backtracking algorithm and place N queens on an $N \times N$ chessboard such that no two queens attack each other.

Description :

The N-Queens problem is a classic problem in computer science where N queens must be placed on an $N \times N$ chessboard so that no two queens threaten each other.

Backtracking is a problem-solving technique that builds the solution step by step and removes choices that lead to invalid solutions.

For each row, it checks whether placing a queen in a particular column is safe. If the position is safe, the queen is placed and the algorithm moves to the next row. If a conflict occurs, the algorithm backtracks by removing the previously placed queen and tries another position.

Working Principle :

1. Start placing queens from the first row.
2. For each column in the row, check if placing a queen is safe.
3. If safe, place the queen and move to the next row.
4. If no safe position is found, backtrack to the previous row and try another column.
5. Repeat this process until all queens are placed successfully.

Applications :

- Solving constraint satisfaction problems
- Artificial Intelligence and search algorithms

Procedure / Algorithm with Complexities:

1. Start the program.
2. Read the value of N (size of the chessboard).
3. Begin placing queens from row 1.
4. For each column in the current row:
 - Check whether the position is safe (no queen in same column or diagonals).
5. If safe, place the queen and move to the next row.
6. If all queens are placed successfully, display the solution.
7. If no safe position exists, remove the previously placed queen and try the next column.
8. Repeat until all possible solutions are explored.
9. Stop the program.

Time Complexity:

- $O(N!)$ in the worst case.

Space Complexity:

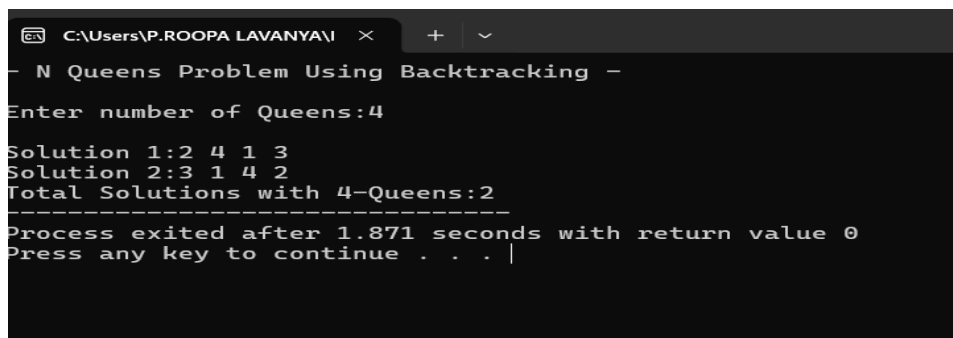
- $O(N)$ for storing queen positions.

Source Code :

```
#include<stdio.h>
```

```
#include<stdlib.h>
int board[20],count;
void queen(int row,int n);
void print(int n);
int place(int row,int column);
int main(){
int n;
printf("- N Queens Problem Using Backtracking -");
printf("\n\nEnter number of Queens:");scanf("%d",&n);
queen(1,n);
printf("\nTotal Solutions with %d-Queens:%d",n,count);return 0;}
void print(int n){
int i;
printf("\nSolution %d:",++count);
for(i=1;i<=n;++i)
printf("%d ",board[i]);}
int place(int row,int column){
int i;
for(i=1;i<=row-1;++i){
if(board[i]==column)return 0;
else if(abs(board[i]-column)==abs(i-row))
return 0;}return 1;}
void queen(int row,int n){
int column;
for(column=1;column<=n;++column){
if(place(row,column)){board[row]=column;
if(row==n)print(n);
else queen(row+1,n);}}}
```

Input & Output:



```
C:\Users\P.ROOPA LAVANYA\ - N Queens Problem Using Backtracking -
Enter number of Queens:4
Solution 1:2 4 1 3
Solution 2:3 1 4 2
Total Solutions with 4-Queens:2
-----
Process exited after 1.871 seconds with return value 0
Press any key to continue . . . |
```

Result :

Thus, the N-Queens problem was successfully implemented using the Backtracking algorithm, and the queens were placed on the chessboard such that no two queens attack each other.

Practice – 5.2

Aim :

To write a program to solve the Travelling Salesperson Problem (TSP) using the Branch and Bound approach and find the minimum cost tour that visits each city exactly once and returns to the starting city.

Description :

The Travelling Salesperson Problem (TSP) is a classic optimization problem in computer science. In this problem, a salesperson must visit N cities exactly once and return to the starting city, while minimizing the total travel cost. The Branch and Bound technique is used to solve this problem more efficiently brute force. The algorithm builds the solution step by step by branching into possible city paths and calculating a lower bound cost for each path. If the cost of a partial path exceeds the current minimum cost, that path is pruned (discarded).

Working Principle :

1. Start from the initial city.
2. Generate possible paths to unvisited cities.
3. Calculate the lower bound cost for each path.
4. Discard paths whose cost is greater than the current minimum cost.
5. Continue exploring the remaining paths.
6. Finally, obtain the minimum cost tour that visits all cities once and returns to the starting city.

Applications :

- Route optimization in transportation systems
- Circuit board design

Procedure / Algorithm with Complexities :

1. Start the program.
2. Read the number of cities n.
3. Input the cost matrix representing distances between cities.
4. Start from the first city and mark it as visited.
5. Generate possible paths by visiting unvisited cities.
6. Calculate the lower bound cost for each path.
7. Compare the bound with the current minimum cost.
8. If the bound is less than the current minimum, continue exploring the path.
9. Otherwise, prune the path.
10. Repeat the process until all possible paths are evaluated.
11. Display the minimum cost tour.
12. Stop the program.

Time Complexity: $O(n!)$ in the worst case.

Space Complexity: $O(n^2)$ to store the cost matrix.

Source Code:

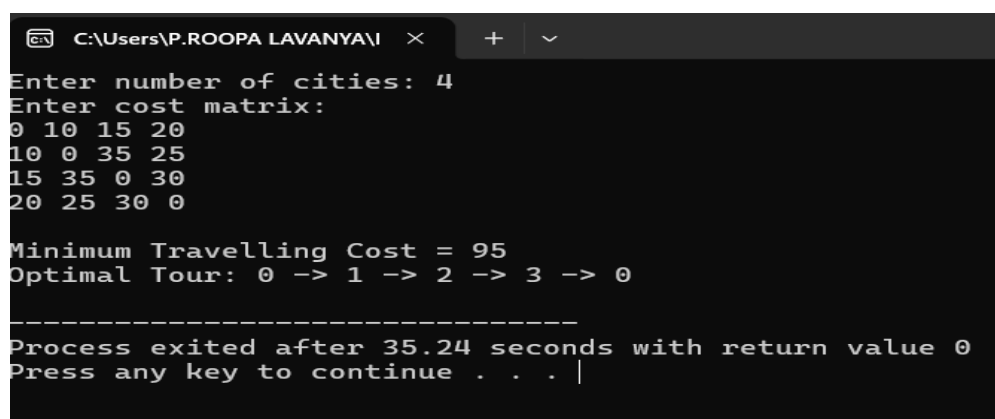
```
#include<stdio.h>
#include<limits.h>
#define MAX 10
```

```

int cost[MAX][MAX];
int visited[MAX];int path[MAX],bestPath[MAX];int n,i,j;
int minCost=INT_MAX;
void tsp(int city,int count,int currentCost);
int main(){ printf("Enter number of cities: ");scanf("%d",&n);
printf("Enter cost matrix:\n");
for(i=0;i<n;i++){
for(j=0;j<n;j++){scanf("%d",&cost[i][j]);}}
visited[0]=1;path[0]=0;tsp(0,1,0);
printf("\nMinimum Travelling Cost = %d\n",minCost);printf("Optimal Tour: ");
for(i=0;i<n;i++) printf("%d -> ",bestPath[i]);printf("0\n");return 0;}
void tsp(int city,int count,int currentCost){
if(count==n){
int total=currentCost+cost[city][0];
if(total<minCost){
minCost=total;
for(i=0;i<n;i++){
bestPath[i]=path[i];}return;}
for(i=0;i<n;i++){
if(!visited[i]&&cost[city][i]!=0){
int tempCost=currentCost+cost[city][i];
if(tempCost<minCost){
visited[i]=1;path[count]=i;
tsp(i,count+1,tempCost);
visited[i]=0;}}}
}
}

```

Input & Output :



```

C:\Users\P.ROOPA LAVANYA\I
Enter number of cities: 4
Enter cost matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Minimum Travelling Cost = 95
Optimal Tour: 0 -> 1 -> 2 -> 3 -> 0

-----
Process exited after 35.24 seconds with return value 0
Press any key to continue . . . |

```

Result :

Thus, the Travelling Salesperson Problem was successfully implemented using the Branch and Bound approach, and the minimum cost tour for visiting all cities and returning to the starting city was obtained.